

tmx Clipboard — Operator Guide

Revision: 2 **Last modified:** 2026-05-29T11:40:00Z **Authority:** vasic-digital tmux project **Maintainer:** milosvasic **Scope:** Operator-facing guide for the v1.0.14 / v1.0.15 / v1.0.17 clipboard surface — multi-line copy-OUT, paste-IN, modifier-drag selection, and the prefix `m` mouse-toggle escape hatch inside mouse-tracking TUIs (especially Claude Code, vim, less, htop).

1. Quick recipe (TL;DR)

Goal	Keystroke / mouse
Select & copy a few lines of plain shell output	drag with mouse, release — then <code>pbpaste / wl-paste / xclip -o</code>
Select & copy inside Claude Code, vim, less, htop	Shift-drag (works on every terminal) — then paste as above
Just let me select with the mouse like a normal terminal	prefix + m to toggle tmux mouse OFF, then drag normally and <code>Cmd-C</code> / right-click → Copy; prefix + m again to restore tmux scrollbar
Copy without a mouse	<code>prefix + [→ v → arrows / j / k → y</code>
Paste OS clipboard INTO the current pane	<code>prefix + P</code> (capital P; lowercase p is previous-window)
Scroll back through history	wheel up (any device, including Termux touch)

Replace `prefix` with your tmux prefix (default `C-b`).

2. Inside Claude Code (and other mouse-tracking TUIs)

The forensic anchor for this section is the 2026-05-28 operator mandate: *"Selecting multiple lines and copying of them does not work. We MUST BE able to scroll vertically everywhere and copy / paste anything! Especially in Claude Code (c`laude` command)!"*

Why plain mouse-drag does NOT enter copy-mode in Claude Code

Claude Code, like vim / less / htop / fzf, runs on the terminal's **alternate screen** and requests **mouse tracking** (DECSET 1000/1002/1003/1006). When mouse tracking is active, tmux's default MouseDrag1Pane binding forwards every drag event to the application via send -M so the TUI can handle its own click behaviour. The drag therefore never reaches tmux's copy-mode, so the obvious select-with-mouse motion appears to do nothing for the operator.

This is by design upstream — but for the operator who wants to copy multiple lines out of Claude Code's transcript, it is the worst possible UX.

The escape: Alt-drag (macOS) or Shift-drag (Linux)

tmux 3.6a supports modifier-prefixed mouse events (M- for Alt/meta, S- for Shift; see man tmux under "key names"). vasic-digital tmux v1.0.15 adds two dedicated bindings to the shipped ~/.tmux.conf:

```
bind -n M-MouseDrag1Pane copy-mode -M
bind -n S-MouseDrag1Pane copy-mode -M
bind -T copy-mode-vi M-MouseDragEnd1Pane send-keys -X copy-pipe-and-cancel "#{@clip}"
bind -T copy-mode-vi S-MouseDragEnd1Pane send-keys -X copy-pipe-and-cancel "#{@clip}"
```

Effect: **hold Alt (Option on macOS) or Shift while you drag** and tmux forces copy-mode entry regardless of what the TUI is doing with the mouse. Release the drag and the selection goes straight through the OS clipboard pipeline (@clip, see §3 below). A plain (unmodified) drag still goes to the app, so Claude Code's own click behaviour is preserved.

OS	Recommended modifier	Why
Any OS / any terminal	Shift-drag	The most reliable in-tmux gesture: no mainstream terminal claims Shift as a bypass modifier, so Shift-mouse events are forwarded to tmux, which forces copy-mode via S-MouseDrag1Pane. Works on macOS AND Linux.
macOS (Terminal.app, WezTerm, Ghostty, Alacritty)	Alt-drag (hold Option) also works	Option passes through to tmux on these terminals → M-MouseDrag1Pane
Linux (gnome-terminal, konsole, xterm, kitty, foot, alacritty)	Alt-drag also works	Option/Meta forwarded to tmux on X11/Wayland
Termux on Android	long-press → "Select text" (terminal-native), then drag	Termux's gesture handler intercepts before tmux sees the event

iTerm2 caveat (forensic anchor: user report 2026-05-29). With iTerm2's default *Option Key Sends = Normal*, iTerm2 treats **Option-drag as its OWN native-selection bypass** — the drag never reaches tmux, so M-MouseDrag1Pane does NOT fire. That native iTerm2 selection still works (copy it with Cmd-C), but if you want tmux's copy-mode selection, **use Shift-drag** (or the prefix m toggle below). This is why Shift is the recommended cross-terminal gesture.

The simplest, always-works escape: prefix + m (v1.0.17)

If you just want to **select and copy with the mouse like a normal terminal** — no modifier gymnastics, works identically in Claude Code, vim, plain shell, every terminal:

```
bind m set -g mouse \; display-message 'mouse #{?mouse,ON ...,OFF
(use native terminal selection: drag + Cmd-C)}'
```

Press **prefix m** to toggle tmux's mouse handling **OFF**. With mouse off, tmux stops capturing drags entirely, so the **outer terminal's own selection** (iTerm2 / Terminal.app / WezTerm / any Linux terminal: click-drag → Cmd-C or right-click → Copy) works **everywhere, including inside Claude Code**. Press prefix m again to turn mouse back ON and restore tmux scrollbar + copy-mode. The status line shows the new state each time. (tmux toggles a flag option when its value is omitted — see Sources verified.)

3. On the keyboard (no mouse at all)

Vi mode in copy-mode is enabled by default (set -g mode-keys vi in the shipped config).

prefix + [	enter copy-mode at the bottom of the buffer
prefix + PageUp	enter copy-mode AND page up in one motion
k / j	move up / down one line
C-u / C-d	half-page up / down
g / G	top / bottom of scrollbar
/ / ?	search forward / backward
v	begin selection
V	line-mode selection (whole-line block)
C-v	rectangle (block) selection
y	copy through @clip → OS clipboard, exit copy-mode
Enter	same as y
q / Escape	exit copy-mode without copying

The y binding is what physically reaches pbpaste / wl-paste / xclip -o. Test 44 (scripts/tests/44_clipboard_copy_out_physical.sh) proves the literal y keystroke makes a marker show up on the OS clipboard end-to-end.

4. Paste OS clipboard INTO the current pane (v1.0.15)

set -g set-clipboard external is COPY-OUT only — OSC-52 is not standardised for paste-IN. v1.0.15 adds an explicit binding:

```
set -g @clip-read 'sh -c "command -v pbpaste >/dev/null 2>&1 &&
exec pbpaste; \
                                command -v wl-paste >/dev/null 2>&1 &&
exec wl-paste -n; \
                                command -v xclip >/dev/null 2>&1 && exec
xclip -o -selection clipboard; \
                                command -v termux-clipboard-get >/dev/
null 2>&1 && exec termux-clipboard-get; \
                                printf "\"\"'"
bind P run -b 'tmux load-buffer - <<< "${#@clip-read})" \; tmux
paste-buffer -p'
```

- prefix + P (capital) reads the OS clipboard via the OS-adaptive @clip-read helper and pastes it into the current pane.
- paste-buffer -p enables **bracketed-paste** mode (per man tmux: *"If -p is specified, paste bracket control codes are inserted around the buffer if the application has requested bracketed paste mode"*). Apps that understand bracketed paste (bash, zsh, vim, neovim, Claude Code's prompt) treat the inserted text as a single literal block — no accidental command execution from a stray newline.

Lowercase prefix + p is left bound to tmux's previous-window so existing muscle memory is preserved.

5. On headless Linux (no clipboard tool installed)

@clip and @clip-read probe a closed set of tools in this order:

Order	Tool	Provides	Install
1	pbcopy / pbpaste	macOS native	preinstalled on Darwin
2	wl-copy / wl-paste	Wayland (sway, GNOME Wayland, KDE Wayland)	apt install wl-clipboard
3	xclip	X11 (Xorg)	apt install xclip
4	termux-clipboard-set / termux-clipboard-get	Android Termux	pkg install termux-api
5	OSC-52 (set-clipboard external)	any terminal that supports the OSC-52 escape	already enabled by the shipped config

If none of 1–4 are available AND the operator's terminal does not honour OSC-52, copy-OUT physical-proof test (T5 of test 44) **honestly SKIPs** with reason `no_clipboard_tool_reachable`. The binding chain (T3/T4) still verifies end-to-end inside tmux; only the final "did it land in pbpaste" hop is unverifiable on that topology. This is a §11.4.3 topology SKIP, not a PASS-bluff.

6. Troubleshooting

"Copy works in tmux but pbpaste returns the OLD clipboard." The `@clip` user option is set by the shipped `~/ .tmux.conf` — confirm it is present: `tmux show -gv @clip` inside a tmx session. If the output is empty, your `~/ .tmux.conf` is from before v1.0.14; re-run `bash scripts/setup.sh` to regenerate.

"Alt-drag selects nothing; the app still gets the drag." Your terminal is capturing Alt before tmux sees it (common on Linux with gnome-terminal's "Use Alt as Meta" preference disabled). Try Shift-drag instead. As a permanent fix, check the terminal's "Pass Alt through to applications" / "Use Option as Meta key" setting.

"prefix + P does nothing." Either (a) `@clip-read` resolved to an empty probe — install one of `pbpaste` / `wl-paste` / `xclip` / `termux-clipboard-get`; or (b) the OS clipboard is genuinely empty. Verify with `tmux show -gv @clip-read` and then run the value in a shell to see what it returns.

"Mouse buttons behave weirdly after an aborted Alt-drag." Press q or Escape to exit copy-mode cleanly. tmux returns the pane to the app's mouse mode on copy-mode exit.

"Scrollbar works in the shell but not in Claude Code." That's the same root cause as the selection problem above — Claude Code holds the alt-screen with mouse tracking. The shipped config overrides `WheelUpPane` / `WheelDownPane` so wheel scroll ALWAYS drives tmux's scrollbar regardless of the app's mouse mode. If wheel scroll still misbehaves, `tmux show -g | grep WheelUp` should show the binding; if it doesn't, regenerate the config.

7. Anti-bluff verification (positive evidence on every PASS)

Test	What it proves	Where
44 — clipboard copy-OUT physical	y keystroke through @clip reaches the OS clipboard; SKIPs honestly on headless	scripts/tests/ 44_clipboard_copy_out_physical.sh
45 — multi-line copy regression	keyboard v + arrows + y selects ≥ 3 lines; round-trip via pbpaste	scripts/tests/ 45_multi_line_keyboard_copy.sh
46 — paste-IN physical		scripts/tests/ 46_paste_in_physical.sh

Test	What it proves	Where
	prefix + P reads @clip-read and inserts into the pane	
47 — mouse- surface confirmation	tmux observes the M- / S- modifier on drag-start	scripts/tests/ 47_modifier_mouse_surface.sh
48 — modifier- drag binding chain	M-MouseDrag1Pane + M- MouseDragEnd1Pane resolves to @clip end-to-end	scripts/tests/ 48_modifier_drag_binding_chain.sh

Each test follows Constitution §101 (positive captured evidence) and §11.4.69 (sink-side / downstream evidence — pbpaste IS the sink). Paired mutations M44 / M46 / M48 in `meta_test_false_positive_proof.sh` strip the relevant binding and assert each test FAILs (CAUGHT + FEATURE RESTORED).

8. Cross-references

- `scripts/tmux.conf.template` — the canonical source of all bindings cited above
- `docs/guide/README.md` §5 — operator-command reference
- `docs/scrolling/README.md` — sister guide for wheel / touch scrolling in Claude Code
- `Fixed.md` A35 (v1.0.14 copy-OUT) + A37 (v1.0.15 multi-line + paste-IN + modifier-drag)
- `CHANGELOG.md` v1.0.14 + v1.0.15 sections

Sources verified 2026-05-29

- **tmux upstream man page** — <https://man.openbsd.org/tmux.1> (re-verified 2026-05-29 via WebFetch, OpenBSD ships the upstream tmux man page). Confirmed for the v1.0.17 prefix m addition: modifier prefixes "Ctrl keys may be prefixed with C- or ^, Shift keys with S- and Alt (meta) with M-" (validates S-MouseDrag1Pane / M-MouseDrag1Pane); -M "passes through a mouse event (only valid if bound to a mouse key binding, see MOUSE SUPPORT)"; flag/choice options "can be omitted ... to toggle its value" (validates set -g mouse with no value as a toggle); #{?condition,true,false} is the standard FORMATS conditional used in the display-message status hint.

Sources verified 2026-05-28

- **tmux 3.6a man page** — `man tmux` on the host (Mistborn Darwin arm64, tmux 3.6a from `tmux/build-darwin/bin/tmux`). Confirmed: S- / M- modifier

prefixes for keys including mouse events ("Ctrl keys may be prefixed with C- or ^, Shift keys with S- and Alt (meta) with M-"); MouseDrag1 / MouseDragEnd1 events listed in MOUSE SUPPORT; paste-buffer -p documented as "paste bracket control codes are inserted around the buffer if the application has requested bracketed paste mode"; user options prefixed with @ are arbitrary and string-typed.

- **tmux upstream man page mirror** — <https://man.openbsd.org/tmux.1> (verified 2026-05-28; OpenBSD ships the upstream tmux man page).
- **Anthropic Claude Code documentation** — <https://code.claude.com/docs/en/> (verified 2026-05-28; the public docs do not currently publish a tmux-integration page, so the alt-screen + mouse-tracking behaviour cited above is documented here from direct observation against the Claude Code CLI v2.x — the upstream binary that ships as bin/claude.exe Mach-O on macOS and is exercised in scripts/tests/47..48).
- **Synthetic alt-screen surrogate** — scripts/tests/helpers/synthetic_alt_screen_app.py (this repo) substitutes for Claude Code in CI to avoid §11.4.98 OAuth/interactive flake while still exercising the exact alt-screen + mouse-tracking surface.